

2025년 2회 정보처리기사 실기 복원 문제 (해설 추가본)

출처: <https://chobopark.tistory.com/554> (Life-Journey)

1번

다음은 파일 구조와 관련된 설명이다. 괄호 안에 들어갈 가장 알맞은 용어를 작성하시오.

데이터베이스의 물리 설계 시 레코드 접근 방법은 순차 접근, [] 방법, 해싱 방법 등이 있다. 이 중 [] 방법은 레코드의 키 값과 포인터를 쌍으로 묶어 저장하며, 검색 시 키 값을 기준으로 빠르게 탐색하도록 설계되어 있다. (<키 값, 포인터> 쌍으로 구성된 자료구조 사용)

접근 방법	설명
순차 접근	레코드를 처음부터 하나씩 검사
[] 접근	키-값 쌍으로 구성되어 빠르게 검색
해싱 접근	해시 함수를 이용해 직접 주소 계산 후 접근

정답: 인덱스 (또는 색인)

해설:

인덱스(Index, 색인)는 책 뒤의 '찾아보기'처럼, 데이터를 빠르게 찾기 위해 <키 값, 포인터> 쌍을 따로 모아둔 자료구조입니다.

- 순차 접근: 처음부터 하나씩 다 확인 → 느림.
- 인덱스 접근: 키 값으로 정렬된 색인을 보고 원하는 데이터의 위치(포인터)로 바로 점프 → 빠름.
- 해싱 접근: 해시 함수로 주소를 직접 계산 → 매우 빠름(평균 $O(1)$).

책에서 특정 단어를 찾을 때 첫 페이지부터 넘기는 대신(순차), 찾아보기 색인에서 페이지 번호를 보고 바로 펴는 것(인덱스)을 떠올리면 됩니다.

2번

다음은 데이터베이스 릴레이션의 구성 요소 중 하나에 대한 설명이다. 보기에서 알맞은 기호를 골라 작성하시오.

릴레이션에서 열(Column)을 의미하며 데이터 항목의 속성 또는 특성을 나타낸다. 파일 구조에서의 필드(Field)에 해당하며 릴레이션에서 행(Tuple)의 구성 요소가 된다.

[보기] ㉠. Cardinality ㉡. Domain ㉢. Attribute ㉣. Degree ㉤. Schema ㉥. Tuple

정답: ㉢. Attribute

해설:

속성(Attribute)은 릴레이션(테이블)에서 열(Column)을 의미합니다.

- 예를 들어 '학생' 테이블에 이름, 나이, 학번 같은 열이 있다면 각각이 하나의 속성(Attribute)입니다.
- 파일 구조의 필드(Field)와 같은 개념입니다.
- 하나의 행(튜플)은 여러 속성 값들이 모여 구성됩니다.

비교: Attribute(속성)=열 / Tuple(튜플)=행 / Degree(차수)=속성의 개수 / Cardinality(카디널리티)=튜플의 개수 / Domain(도메인)=한 속성이 가질 수 있는 값의 범위.

"열 = 속성(Attribute)"으로 기억하세요.

3번

다음은 정보보안 관련 문제이다. 알맞은 단어를 작성하시오.

원격 접속과 관련된 보안 프로토콜이며 암호화된 통신을 제공한다. 공개키 기반 인증 방식을 사용하고 기본 포트는 22번이며, Telnet의 보안 취약점을 보완한 대안으로 널리 사용된다.

정답: SSH

해설:

SSH(Secure SHell)는 원격 컴퓨터에 **안전하게(암호화하여)** 접속하기 위한 프로토콜입니다.

- 핵심 키워드: **기본 포트 22번, 암호화 통신, 공개키 인증.**
- 옛 방식인 Telnet은 비밀번호와 데이터를 암호화하지 않아 노출 위험이 컸는데, SSH가 이를 암호화로 보완했습니다.

"포트 22 + Telnet 보완 = SSH"라는 키워드만 잡으면 됩니다. 매우 자주 출제되는 단골 문제입니다.

4번

스케줄링 알고리즘에 관한 설명을 읽고 (1)과 (2)에 알맞은 명칭을 쓰시오.

- (1) CPU burst 시간이 짧은 프로세스를 우선적으로 처리하는 방식. "Shortest Next CPU Burst"라고도 불린다.
- (2) 위 방식을 선점형으로 구현한 형태로, 실행 중인 프로세스보다 더 짧은 burst 시간을 가진 프로세스가 도착하면 CPU를 선점한다.

정답: (1) SJF (Shortest Job First) / (2) SRT (Shortest Remaining Time)

해설:

실행 시간이 짧은 작업을 먼저 처리하는 두 스케줄링 방식입니다. 차이는 **선점(빼앗기) 여부**입니다.

- **SJF(Shortest Job First): 실행 시간이 가장 짧은** 프로세스를 먼저 처리. **비선점형**이라 한 번 시작하면 끝까지 실행합니다.
- **SRT(Shortest Remaining Time): SJF의 선점형 버전**입니다. 실행 중이라도 **남은 시간이 더 짧은** 프로세스가 도착하면 CPU를 빼앗깁니다.

'비선점'은 한 번 잡으면 안 뺏김, '선점'은 더 급한(짧은) 게 오면 양보. SJF는 "가장 짧은 작업 먼저", SRT는 "남은 시간이 가장 짧은 것 먼저(선점)"로 구분하세요.

5번

다음은 Java의 문제이다. 알맞는 출력값을 작성하시오.

```
public class Main {
    public static void change(String[] data, String s){
        data[0] = s;
        s = "Z";
    }

    public static void main(String[] args) {
        String data[] = { "A" };
        String s = "B";
        change(data, s);
        System.out.print(data[0] + s);
    }
}
```

정답: BB

해설:

자바의 **매개변수 전달 방식**이 핵심입니다. "배열(객체)은 참조가 전달되고, String 변수 자체의 재할당은 원본에 영향이 없다"는 점을 구분해야 합니다.

change 함수 안에서:

- data[0] = s → data는 **배열 객체의 참조**라서, 함수 안에서 data[0]을 바꾸면 **원본 배열의 0번 요소가 실제로 "B"로 바뀝니다.**
- s = "Z" → 이건 함수 안의 **지역 변수 s**에 "Z"를 새로 대입하는 것일 뿐, main의 s에는 영향이 없습니다.

따라서 main으로 돌아오면 data[0]은 "B"로 바뀌고, main의 s는 여전히 "B".

출력 = data[0] + s = "B" + "B" = "**BB**"

핵심: 배열 요소를 직접 수정하면 원본이 바뀌지만, 매개변수 변수 자체에 새 값을 대입하면 원본은 그대로입니다.

6번

다음은 IP 주소와 서브넷 마스크에 관한 문제이다. 괄호 안에 들어갈 알맞은 값을 쓰시오.

호스트 IP 223.13.234.132, 서브넷 마스크 255.255.255.192일 때, 이 호스트가 속한 네트워크 주소는 223.13.234.(①)이고, 사용 가능한 호스트 수는 (②)개이다. (네트워크 주소와 브로드캐스트 주소 제외)

정답: ① 128 / ② 62

해설:
서브넷 계산 문제입니다. 마지막 옥텟에 집중하면 됩니다.

- 서브넷 마스크 255.255.255.**192**를 분석합니다.
- $192 = 2진수\ 11000000$. 상위 2비트가 네트워크, 하위 6비트가 호스트용입니다.
 - 한 네트워크의 크기 = $256 - 192 = 64$ (마지막 옥텟이 64씩 묶임).
 - 네트워크 경계: 0, 64, 128, 192.

호스트 IP의 마지막 옥텟은 132입니다. 132는 **128~191** 범위에 속하므로 네트워크 주소는 마지막 옥텟이 **128**입니다. → ① 128 (즉 223.13.234.128)

- 사용 가능한 호스트 수:
- 호스트 비트가 6개 → $2^6 = 64$ 개 주소.
 - 그중 네트워크 주소(맨 처음)와 브로드캐스트 주소(맨 끝)를 빼면 → $64 - 2 = 62$ 개. → ②

공식: 호스트 수 = $2^{(호스트\ 비트\ 수)} - 2$. 네트워크 블록 크기로 어느 묶음에 속하는지 찾는 것이 핵심입니다.

7번

다음은 디자인 패턴에 관한 문제이다. 알맞는 단어를 작성하시오.

어떤 객체에 대한 접근을 제어하거나 추가 기능을 부여하기 위해 대리 객체를 사용하는 디자인 패턴이다. 실제 객체 생성을 지연시켜 메모리·자원을 절약하고 정보 은닉을 강화할 수 있다.

정답: Proxy

해설:
Proxy(프록시) 패턴은 실제 객체에 대한 접근을 중간에서 대신 처리하는 '**대리인**' 객체를 두는 구조 패턴입니다.

- 실제 객체를 직접 부르지 않고 대리 객체를 거치게 하여, ① 접근 제어, ② **객체 생성 지연**(필요할 때만 생성 → 메모리 절약), ③ 부가 기능 추가가 가능합니다.
- 무거운 객체를 미리 만들지 않고, 실제로 필요한 순간에만 만드는 '지연 생성'이 대표 용도입니다.

키워드 "대리 객체 + 접근 제어 + 생성 지연 = Proxy"입니다. 구조 패턴에 속합니다.

8번

다음은 웹 데이터 교환 방식에 관한 문제이다. 괄호 안에 들어갈 알맞은 용어를 작성하시오.

()은/는 웹 페이지 전체를 다시 불러오지 않고 JavaScript와 XML(또는 JSON)을 이용하여 일부 콘텐츠만 비동기적으로 갱신할 수 있는 기술이다.

정답: AJAX

해설:
AJAX(Asynchronous JavaScript and XML)는 **페이지 전체를 새로고침하지 않고**, 자바스크립트가 백그라운드에서 데이터만 주고받아 **화면 일부만 갱신**하는 기술입니다.

- '비동기(Asynchronous)'가 핵심입니다. 서버에 요청을 보낸 뒤 응답을 기다리는 동안에도 다른 작업을 계속할 수 있습니다.
- 예전엔 데이터 하나 바꾸려 해도 화면 전체가 깜빡였지만, AJAX 덕분에 부드럽게 일부만 바뀝니다. (예: SNS의 무한 스크롤)

키워드 "비동기 + 일부만 갱신 + 페이지 전체 새로고침 없음 = AJAX"입니다.

9번

다음은 Java 언어의 문제이다. 알맞는 출력값을 작성하시오.

```
public class Main {
    static interface F {
        int apply(int x) throws Exception;
    }
}
```

```

public static int run(F f) {
    try {
        return f.apply(3);
    } catch (Exception e) {
        return 7;
    }
}

public static void main(String[] args) {
    F f = (x) -> {
        if (x > 2) {
            throw new Exception();
        }
        return x * 2;
    };
    System.out.print(run(f) + run((int n) -> n + 9));
}
}

```

정답: 19

해설:

람다식(Lambda)과 예외 처리가 결합된 문제입니다. 람다식은 함수를 간단히 표현하는 문법으로, 여기서는 인터페이스 F의 apply 메서드를 구현합니다.

run 함수는 항상 f.apply(3) 을 호출합니다(인자로 3을 줌). 예외가 나면 7을 반환합니다.

두 번의 run 호출을 봅시다:

- run(f) : f는 if(x>2) throw 인 람다. apply(3) → 3>2이므로 **예외 발생** → catch에서 **7** 반환.
- run((int n) -> n + 9) : 이 람다는 n + 9 . apply(3) → 3+9 = **12** (예외 없음).

합 = 7 + 12 = **19**

핵심: run은 무조건 인자 3으로 apply를 호출하고, 첫 람다는 3에서 예외(→7), 둘째 람다는 3+9(→12)를 반환합니다.

10번

다음은 Java 언어의 문제이다. 알맞는 출력값을 작성시오.

```

public class Main {
    public static class Parent {
        public int x(int i) { return i + 2; }
        public static String id() { return "P"; }
    }

    public static class Child extends Parent {
        public int x(int i) { return i + 3; }
        public String x(String s) { return s + "R"; }
        public static String id() { return "C"; }
    }

    public static void main(String[] args) {
        Parent ref = new Child();
        System.out.println(ref.x(2) + ref.id());
    }
}

```

정답: 5P

해설:

인스턴스 메서드(오버라이딩)와 **static 메서드**의 호출 규칙 차이가 핵심입니다.

- ref 는 선언 타입 Parent, 실제 객체 Child입니다.
- ref.x(2) : x(int)는 **인스턴스 메서드**이고 **오버라이딩**되었습니다. 인스턴스 메서드는 **실제 객체(Child)** 기준 → Child의 x(2) = 2+3 = **5**. (동적 바인딩)
- ref.id() : id()는 **static(정적) 메서드**입니다. static 메서드는 **선언 타입(Parent)** 기준으로 호출됩니다 → Parent의 id() = **"P"**. (정적 바인딩, 오버라이딩)

안 됨)

합 = 5 + "P" = "5P"

핵심 한 줄: "인스턴스 메서드는 실제 객체, static 메서드는 선언 타입"을 따릅니다. static 메서드는 오버라이딩 개념이 적용되지 않는다는 점이 함정입니다.

11번

다음 아래 제어 흐름 그래프가 분기 커버리지를 만족하기 위한 테스트 순서를 쓰시오. (※ 원문에 그래프 이미지 포함)

정답: 1234561, 124567 (또는 1234567, 124561)

해설:

분기 커버리지(Branch Coverage) = 결정 커버리지는 모든 분기점의 참/거짓 경로를 최소 한 번씩 실행하는 테스트 기준입니다.

- 제어 흐름 그래프에서 조건문(분기점)마다 갈라지는 두 갈래(참/거짓)가 있습니다.
- 분기 커버리지 100%를 위해, 두 개의 테스트 경로를 골라 **모든 갈림길을 빠짐없이** 지나가게 만듭니다.
- 정답의 두 경로는 각기 다른 갈래를 선택해 합치면 모든 분기를 덮습니다.

커버리지 강도 순서: 문장 < 분기(결정) < 조건 < 조건/결정 < MC/DC < 다중조건. 분기 커버리지는 "갈림길 모두 한 번씩"이라고 기억하세요.

12번

다음은 C언어의 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>
#define SIZE 3

typedef struct {
    int a[SIZE];
    int front;
    int rear;
} Queue;

void enq(Queue* q, int val){
    q->a[q->rear] = val;
    q->rear = (q->rear + 1) % SIZE;
}

int deq(Queue* q) {
    int val = q->a[q->front];
    q->front = (q->front + 1) % SIZE;
    return val;
}

int main() {
    Queue q = {{0}, 0, 0};
    enq(&q,1); enq(&q,2); deq(&q); enq(&q, 3);
    int first = deq(&q);
    int second = deq(&q);
    printf("%d 그리고 %d", first, second);
    return 0;
}
```

정답: 2 그리고 3

해설:

원형 큐(Circular Queue)의 동작을 묻는 문제입니다. 큐는 **먼저 들어간 것이 먼저 나오는(FIFO)** 자료구조입니다.

- enq (삽입)는 rear 위치에 값을 넣고 rear를 한 칸 이동(% SIZE 로 원형 처리).
- deq (꺼내기)는 front 위치의 값을 꺼내고 front를 한 칸 이동.

초기: front=0, rear=0, 배열 = {0,0,0}

연산을 추적합니다:

- enq(1): a[0]=1, rear=1
- enq(2): a[1]=2, rear=2
- deq(): a[0]=1 꺼냄, front=1 (1이 빠짐)
- enq(3): a[2]=3, rear=0
- first = deq(): a[1]=2 꺼냄, front=2
- second = deq(): a[2]=3 꺼냄, front=0

출력: "2 그리고 3". 먼저 넣은 1은 이미 빠졌으니, 남은 것을 순서대로 꺼내면 2, 3입니다. FIFO 원칙을 기억하세요.

13번

라운드로빈(RR) 방식을 이용하고 아래 내용을 참고하여 평균대기시간을 구하시오. (시간 할당량 4ms, 컨텍스트 스위칭 시간 무시)

프로세스	도착 시간(ms)	실행 시간(ms)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

정답: 11.75ms

해설:

라운드로빈(RR)은 각 프로세스에 정해진 시간(타임 퀀텀, 여기서 4ms)씩 번갈아 CPU를 주는 방식입니다. 시간이 다 되면 뒤로 가서 줄을 서고 다음 프로세스에게 넘깁니다.

타임 퀀텀 4로 실행 순서를 추적합니다:

시간 0-4:	P1 실행 (P1 남은 8→4)
시간 4-8:	P2 실행 (P2 남은 4→0 완료, 완료시각 8)
시간 8-12:	P3 실행 (P3 남은 9→5)
시간 12-16:	P4 실행 (P4 남은 5→1)
시간 16-20:	P1 실행 (P1 남은 4→0 완료, 완료시각 20)
시간 20-24:	P3 실행 (P3 남은 5→1)
시간 24-25:	P4 실행 (P4 남은 1→0 완료, 완료시각 25)
시간 25-26:	P3 실행 (P3 남은 1→0 완료, 완료시각 26)

각 프로세스 대기시간 = 완료시각 - 도착시각 - 실행시간:

- P1: 20 - 0 - 8 = 12
- P2: 8 - 1 - 4 = 3
- P3: 26 - 2 - 9 = 15
- P4: 25 - 3 - 5 = 17

평균 = (12 + 3 + 15 + 17) / 4 = 47 / 4 = **11.75ms**

핵심: RR은 타임 퀀텀만큼만 실행 후 다음으로 넘기고, 안 끝난 프로세스는 큐 뒤로 보냅니다. 완료 시각을 정확히 추적하는 것이 관건입니다.

14번

다음은 C언어의 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

struct dat {
    int x;
    int y;
};

int main() {
```

```

struct dat a[] = {{1, 2}, {3, 4}, {5, 6}};
struct dat* ptr = a;
struct dat** pptr = &ptr;
(*pptr)[1] = (*pptr)[2];
printf("%d 그리고 %d", a[1].x, a[1].y);
return 0;
}

```

정답: 5 그리고 6

해설:
 이중 포인터가 나오지만 핵심은 **구조체 배열의 한 요소를 통째로 복사**하는 것입니다.

- `ptr = a` → `ptr`은 배열 `a`를 가리킴.
- `pptr = &ptr` → `pptr`은 `ptr`을 가리킴. 따라서 `*pptr`은 결국 `ptr` (즉 배열 `a`)과 같습니다.
- `(*pptr)[1]`은 `a[1]`과 같고, `(*pptr)[2]`는 `a[2]`와 같습니다.

따라서 `(*pptr)[1] = (*pptr)[2]`는 `a[1] = a[2]`, 즉 **a[2]의 내용 전체({5,6})를 a[1]에 복사**합니다.

- 복사 전 `a[1] = {3, 4}`

- 복사 후 `a[1] = {5, 6}`

출력: `a[1].x = 5, a[1].y = 6` → "5 그리고 6"

핵심: 구조체끼리 `=`로 대입하면 **모든 멤버가 한꺼번에 복사**됩니다. 복잡한 포인터 표기 `(*pptr)[1]`이 사실 `a[1]`이라는 것만 알면 됩니다.

15번

다음은 Java 언어의 문제이다. 알맞는 출력값을 작성하시오.

```

public class Main {
    public static class B0 {
        public int v;
        public B0(int v) {
            this.v = v;
        }
    }

    public static void main(String[] args) {
        B0 a = new B0(1);
        B0 b = new B0(2);
        B0 c = new B0(3);
        B0[] arr = {a, b, c};
        B0 t = arr[0];
        arr[0] = arr[2];
        arr[2] = t;
        arr[1].v = arr[0].v;
        System.out.println(a.v + "a" + b.v + "b" + c.v);
    }
}

```

정답: 1a3b3

해설:
객체 참조가 핵심입니다. 배열에는 객체 자체가 아니라 **객체를 가리키는 참조**가 담깁니다. 따라서 배열 요소를 통해 객체의 값을 바꾸면 원래 변수(`a, b, c`)도 영향을 받습니다.

초기: `a.v=1, b.v=2, c.v=3`. 배열 `arr = [a, b, c]` (각 칸이 `a, b, c` 객체를 가리킴)

연산을 추적합니다:

- `t = arr[0]` → `t`는 `a` 객체를 가리킴.
- `arr[0] = arr[2]` → `arr[0]`은 이제 `c` 객체를 가리킴.
- `arr[2] = t` → `arr[2]`는 `a` 객체를 가리킴. 결국 `arr = [c객체, b객체, a객체]`.
- `arr[1].v = arr[0].v` → `arr[1]`은 `b` 객체, `arr[0]`은 `c` 객체. `b.v = c.v = 3`. 즉 **b.v가 3으로 바뀜**.

이제 `a, b, c`의 값을 봅니다(객체 자체는 그대로, `b`만 값이 바뀜):

- a.v=1 (변화 없음)
- b.v=3 (위에서 바뀜)
- c.v=3 (변화 없음)

출력 = 1 + "a" + 3 + "b" + 3 = "1a3b3"

핵심: 배열에서 자리를 바꿔도 객체 변수 a/b/c 자체는 그대로 자기 객체를 가리킵니다. arr1의 v를 바꾸면 b.v가 실제로 바뀝니다.

16번

다음은 C언어의 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int p;
    struct node* n;
};

int main() {
    struct node a = {1, NULL};
    struct node b = {2, NULL};
    struct node c = {3, NULL};
    a.n = &b; b.n = &c; c.n = NULL;
    c.n = &a; a.n = &b; b.n = NULL;
    struct node* head = &c;
    printf("%d %d %d", head->p, head->n->p, head->n->n->p);
    return 0;
}
```

정답: 3 1 2

해설: 연결 리스트의 포인터 연결을 추적하는 문제입니다. **마지막에 설정된 연결 상태만** 보면 됩니다.

연결 설정 코드를 순서대로 봅시다(나중 것이 덮어씀):

- 첫 줄: a→b, b→c, c→NULL
- 둘째 줄(덮어씀): **c→a, a→b, b→NULL**

따라서 최종 연결은:

- c.n = a (c 다음은 a)
- a.n = b (a 다음은 b)
- b.n = NULL (b가 끝)

즉 리스트는 **c(3) → a(1) → b(2) → NULL**

head = c부터 따라가면:

- head->p = c의 값 = **3**
- head->n->p = c 다음(a)의 값 = **1**
- head->n->n->p = a 다음(b)의 값 = **2**

출력: "3 1 2". 같은 연결을 두 번 설정한 경우 **나중 설정이 유효**하다는 점이 핵심입니다.

17번

다음은 Python 언어의 문제이다. 알맞는 출력값을 작성하시오.

```
lst = [1,2,3]
dst = {i : i * 2 for i in lst}
s = set(dst.values())
lst[0] = 99
dst[2] = 7
```



```
s.add(99)
print(len(s & set(dst.values())))
```

정답: 2

해설:

딕셔너리 컴프리헨션, 집합, 교집합 연산을 다루는 문제입니다. 변수 간 독립성을 정확히 봐야 합니다.

단계별로 추적합니다:

- lst = [1,2,3]
- dst = {i: i*2 for i in lst} → {1:2, 2:4, 3:6}
- s = set(dst.values()) → {2, 4, 6} (이 시점의 값들로 만든 집합. dst가 나중에 바뀌어도 s는 그대로)
- lst[0] = 99 → lst만 바뀜. dst는 이미 만들어졌으므로 영향 없음.
- dst[2] = 7 → dst의 키 2의 값을 4→7로 변경. 이제 dst = {1:2, 2:7, 3:6}
- s.add(99) → s = {2, 4, 6, 99}

이제 마지막 줄:

- set(dst.values()) = {2, 7, 6} (dst의 현재 값들)
- s & set(dst.values()) = {2,4,6,99} ∩ {2,7,6} = {2, 6} (양쪽 모두에 있는 것)
- len(...) = 2

핵심: s는 처음 dst 값으로 만든 뒤 고정되고(dst 변경에 영향 안 받음), 교집합(&)은 두 집합에 **공통으로 있는 원소**만 남깁니다.

18번

다음은 C언어의 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    char c;
    struct node* p;
};

struct node* func(char* s) {
    struct node* h = NULL, *n;
    while(*s) {
        n = malloc(sizeof(struct node));
        n->c = *s++;
        n->p = h;
        h = n;
    }
    return h;
}

int main() {
    struct node* n = func("BEST");
    while(n) {
        putchar(n->c);
        struct node* t = n;
        n = n->p;
        free(t);
    }
    return 0;
}
```

정답: TSEB

해설:

연결 리스트를 **앞쪽 삽입**으로 만들어 문자열을 뒤집는 문제입니다.

func는 문자열을 한 글자씩 읽으며 **새 노드를 맨 앞에 붙입니다**(n->p = h; h = n;). 이 방식은 마지막에 넣은 것이 맨 앞에 오게 되어 결과적으로 순서가 뒤집힙니다.

- "BEST"를 처리하면:
- 'B' 추가 → [B]
 - 'E' 추가(맨 앞) → [E, B]
 - 'S' 추가(맨 앞) → [S, E, B]
 - 'T' 추가(맨 앞) → **[T, S, E, B]**

main의 while문은 맨 앞부터 출력하며 노드를 따라갑니다(`n = n->p`):

- `T → S → E → B → "TSEB"`

핵심: "맨 앞에 새 노드를 붙이는 방식(head insertion)"은 입력 순서를 뒤집는 효과가 있습니다. 스택(LIFO)과 같은 원리입니다.

19번

다음은 TCP 통신 과정에서 발생할 수 있는 보안 취약점에 대한 설명이다. 이를 이용한 공격 기법을 작성하시오.

공격자가 클라이언트 역할로 수많은 SYN 패킷을 서버에 전송한 뒤 마지막 ACK를 고의로 보내지 않아 서버가 연결 대기 상태를 유지하게 만든다. 서버의 연결 대기 큐가 가득 차면서 서비스 거부 상태가 발생한다.

정답: SYN Flooding

해설:
SYN Flooding은 TCP의 **3-way 핸드셰이크** 과정을 악용한 서비스 거부(DoS) 공격입니다.

- 정상적인 TCP 연결은 3단계로 이뤄집니다:
1. 클라이언트 → 서버: SYN ("연결할래?")
 2. 서버 → 클라이언트: SYN/ACK ("좋아, 너도 준비됐어?")
 3. 클라이언트 → 서버: ACK ("응 준비됐어") → 연결 완료

SYN Flooding 공격자는 1단계 SYN만 잔뜩 보내고 **마지막 3단계 ACK를 보내지 않습니다**. 서버는 "곧 ACK가 오겠지" 하며 **연결 대기 상태(반쯤 열린 연결)**를 유지합니다. 이런 대기가 쌓여 **연결 대기 큐가 가득 차면**, 정상 사용자의 연결 요청을 받을 수 없게 됩니다.

키워드 "SYN만 보내고 ACK 안 보냄 + 대기 큐 가득 = SYN Flooding"으로 기억하세요.

20번

다음 테이블에서 π TTL(employee)에 대한 연산 결과 값을 작성하시오. (※ 원문에 employee 테이블 이미지 포함)

정답: TTL / 부장 / 대리 / 과장 / 차장

해설:
 π (파이)는 관계대수의 **프로젝트(Project)** 연산 기호입니다. "특정 열(속성)만 골라서 추출"하는 연산입니다.

- π TTL(employee) 는 employee 테이블에서 **TTL이라는 열만 뽑아내**라는 의미입니다.
- 결과는 TTL 컬럼에 들어있는 값들(부장, 대리, 과장, 차장)이 됩니다.
- 맨 위 "TTL"은 컬럼 이름(헤더)이고, 그 아래가 실제 값들입니다.

- 관계대수 주요 기호 정리:
- **σ (시그마):** select(행 선택) — 조건에 맞는 행 추출.
 - **π (파이):** project(열 선택) — 원하는 열만 추출.
 - **$\bowtie$:** join(조인) — 두 테이블 연결.
 - **$\div$:** division(나누기).

" π = 세로(열) 추출, σ = 가로(행) 추출"로 구분하세요. 참고로 순수 관계대수의 project는 중복을 제거하지만, 시험 답안은 테이블에 보이는 값을 그대로 나열한 형태입니다.